# Python Operators ??? Real-Time Examples

# Python Operators — Real-Time Examples

Practical, real-world applications of Python operators, useful for classroom teaching and intermediate practice.

---

## 1. Arithmetic Operators — Shopping Bill

Imagine an online shopping application.

```
price = 1200
quantity = 3
discount = 10

total = price * quantity
discount_amount = total * discount / 100
final_amount = total - discount_amount

print("Total:", total)
print("Discount:", discount_amount)
print("Final Amount:", final_amount)
```

**Output:**

```
Total: 3600
Discount: 360.0
Final Amount: 3240.0
```

**What operators are used?**

- `*` → Calculate total
- `/` → Calculate percentage
- `-` → Subtract discount

**Real-time use:** E-commerce websites, billing software, POS systems

---

## 2. Arithmetic — Employee Salary

```
basic_salary = 30000
hra = 5000
bonus = 3000
tax = 4000

gross_salary = basic_salary + hra + bonus
net_salary = gross_salary - tax

print("Gross Salary:", gross_salary)
print("Net Salary:", net_salary)
```

**Output:**

```
Gross Salary: 38000
Net Salary: 34000
```

---

## 3. Assignment Operators — Bank Balance

Assignment operators are very useful when a value changes repeatedly.

```
        balance = 50000

        balance += 10000       # Salary credited
        balance -= 5000        # Money withdrawn
        balance -= 2000        # Bill payment

        print("Current Balance:", balance)
```

**Output:**

```
Current Balance: 53000
```

Instead of:

```
        balance = balance + 10000
```

we can write:

```
        balance += 10000
```

**Real-time use:**

- Banking
- Wallet applications
- Shopping cart
- Inventory management

---

## 4. Assignment Operators — Shopping Cart

```
        cart_items = 2

        cart_items += 1       # Customer adds product
        cart_items += 1       # Customer adds another product
        cart_items -= 1       # Customer removes product

        print("Items in Cart:", cart_items)
```

**Output:**

```
Items in Cart: 3
```

---

## 5. Comparison Operators — Student Result

Suppose a college application checks whether a student has passed.

```
        marks = 68

        print("Pass:", marks >= 40)
        print("Distinction:", marks >= 75)
        print("Failed:", marks < 40)
```

**Output:**

```
Pass: True
Distinction: False
Failed: False
```

**Real-time use:**

- Student result systems
- Exam portals
- Employee performance systems
- Eligibility checking

---

## 6. Comparison Operators — Employee Eligibility

```
        age = 25
        experience = 3

        print(age >= 18)
        print(experience >= 2)
```

**Output:**

```
True
True
```

We can use these results with `if`:

```
        age = 25
        experience = 3

        if age >= 18 and experience >= 2:
            print("Eligible for the job")
        else:
            print("Not Eligible")
```

**Output:**

```
Eligible for the job
```

---

## 7. Logical Operators — Login System

This is a very common real-time example.

A user should be allowed to log in only when both username and password are correct.

```
        username = "admin"
        password = "12345"

        entered_username = "admin"
        entered_password = "12345"

        if entered_username == username and entered_password == password:
            print("Login Successful")
        else:
            print("Invalid Username or Password")
```

**Output:**

```
Login Successful
```

> **Important:** `condition1 and condition2` — Both must be True.

---

## 8. Logical Operators — Movie Ticket

Suppose a cinema gives a child discount if the customer is below 12 OR a senior citizen if above 60.

```
        age = 65
```

```
if age < 12 or age > 60:
    print("Eligible for Discount")
else:
    print("Regular Ticket")
```

**Output:**

```
Eligible for Discount
```

---

# 9. Logical Operators — Online Shopping Offer

```
purchase_amount = 2500
is_member = True

if purchase_amount >= 2000 and is_member:
    print("You get an additional 10% discount")
else:
    print("No additional discount")
```

**Output:**

```
You get an additional 10% discount
```

---

# 10. Identity Operators — `is` and `is not`

A practical example is checking for `None`.

```
user = None

if user is None:
    print("User is not logged in")
else:
    print("User is logged in")
```

**Output:**

```
User is not logged in
```

**Why `is`?**

`None` is a special singleton object in Python, so the standard check is:

```
user is None
```

rather than:

```
user == None
```

---

# 11. Identity Operators — Same Shopping Cart

```
cart1 = ["Laptop", "Mouse"]
cart2 = cart1

print(cart1 is cart2)
```

**Output:**

```
True
```

Both variables refer to the same list object.

Now:

```
cart1 = ["Laptop", "Mouse"]
cart2 = ["Laptop", "Mouse"]

print(cart1 == cart2)
print(cart1 is cart2)
```

**Output:**

```
True
False
```

**Important concept:**

- `==` → Are the values the same?
- `is` → Are they the same object?

---

## 12. Membership Operators — Course Registration

Suppose a student has registered for several courses.

```
courses = ["Python", "React", "JavaScript", "SQL"]

course = "Python"

if course in courses:
    print("Course is registered")
else:
    print("Course is not registered")
```

**Output:**

```
Course is registered
```

---

## 13. Membership Operators — User Role

```
roles = ["admin", "manager", "hr", "developer"]

user_role = "developer"

if user_role in roles:
    print("Valid Role")
else:
    print("Invalid Role")
```

**Output:**

```
Valid Role
```

**Real-time use:**

- Login systems
- Authorization
- E-commerce
- Course registration
- Search systems

---

## 14. Membership Operator — Search in a Product Name

```
        product = "Samsung Galaxy Smartphone"

        if "Samsung" in product:
            print("Samsung product found")
```

**Output:**

```
Samsung product found
```

This type of operation is commonly used in search/filter functionality.

---

# 15. Bitwise Operators — Permission System

Bitwise operators are more advanced but have real-world applications in permissions, flags, networking, and low-level programming.

Suppose:

```
Read    = 1
Write   = 2
Delete  = 4
```

We can combine permissions.

```
        READ = 1
        WRITE = 2
        DELETE = 4

        user_permission = READ | WRITE

        print(user_permission)
```

**Output:**

```
3
```

Because:

```
001
010
---
011 = 3
```

The user has:

- Read ✓
- Write ✓
- Delete ✗

---

# 16. Bitwise AND — Check Permission

```
        READ = 1
        WRITE = 2
        DELETE = 4

        user_permission = READ | WRITE

        if user_permission & WRITE:
            print("User can write")
        else:
            print("Write permission denied")
```

**Output:**

```
User can write
```

This is a good intermediate real-world example of bitwise operators.

---

## 17. Bitwise Shift — Multiplication by 2

```
        number = 10

        result = number << 1

        print(result)
```

**Output:**

```
20
```

Conceptually, `10 << 1` means shift the binary bits one position to the left.

Similarly:

```
        number = 20

        result = number >> 1

        print(result)
```

**Output:**

```
10
```

This is useful for understanding binary operations and performance-oriented programming.

---

## ⭐ One Complete Real-Time Example

For your students, I would recommend giving them one mini project that combines several operators.

### Student Eligibility System

```
        name = "Rahul"
        age = 21
        marks = 78
        attendance = 85
        courses = ["Python", "JavaScript", "React"]

        # Comparison + Logical Operators
        if age >= 18 and marks >= 40 and attendance >= 75:
            print(name, "is eligible for certification")
        else:
            print(name, "is not eligible")

        # Membership Operator
        if "Python" in courses:
            print("Python course is registered")

        # Arithmetic Operators
        total_marks = 100
        percentage = (marks / total_marks) * 100

        print("Percentage:", percentage)

        # Assignment Operator
        bonus_marks = marks
        bonus_marks += 5
```

```
        print("Marks after bonus:", bonus_marks)
```

**Output:**

```
Rahul is eligible for certification
Python course is registered
Percentage: 78.0
Marks after bonus: 83
```

**Operators used:**

| Category | Operators |
|---|---|
| Arithmetic | `/ *` |
| Assignment | `+=` |
| Comparison | `>=` |
| Logical | `and` |
| Membership | `in` |

This is a much better intermediate classroom example because students can see how operators work together in an actual application rather than learning each operator in isolation.